

Devoir surveillé d'informatique

⚠ Consignes

- Les programmes demandés doivent être écrits en C, on suppose que les bibliothèques standard usuelles (`<stdio.h>`, `<stdlib.h>`, `<stdbool.h>`, `<assert.h>`, ...) sont déjà importées.
- On pourra toujours librement utiliser une fonction demandée à une question précédente même si cette question n'a pas été traitée.
- Veillez à présenter vos idées et vos réponses partielles même si vous ne trouvez pas la solution complète à une question.
- La clarté et la lisibilité de la rédaction et des programmes sont des éléments de notation.

□ Exercice 1 : Questions de cours

On considère l'algorithme suivant :

Algorithme : Multiplier sans utiliser *

Entrées : $n \in \mathbb{N}, m \in \mathbb{N}$

Sorties : nm

```

1  r ← 0
2  tant que m > 0 faire
3      m ← m - 1
4      r ← r + n
5  fin
6  return r

```

Q1– Donner les valeurs successives prises par les variables m , n et r si on fait fonctionner cet algorithme avec $n = 7$ et $m = 4$. On pourra recopier et compléter le tableau suivant :

	n	m	r
valeurs initiales	7	4	0
après un tour de boucle	7	3	7
après deux tours de boucle	7	2	14
après trois tours de boucle	7	1	21
après quatre tours de boucle	7	0	28

Q2– Donner une implémentation de cet algorithme en langage C sous la forme d'une fonction `multiplie` de signature `int multiplie(int n, int m)`. On précisera soigneusement la spécification de cette fonction en commentaire dans le code et on vérifiera les préconditions à l'aide d'instructions `assert`.

```

1  // Prends en entrée deux entiers positifs n et m et renvoie leur produit nm
2  int multiplie(int n, int m)
3  {
4      assert(n >= 0 && m >= 0);
5      int r = 0;
6      while (m > 0)
7      {
8          m = m - 1;
9          r = r + n;
10     }
11     return r;
12 }

```

Q3– Donner la définition d'un variant de boucle, puis prouver que cet algorithme termine.

Un *variant de boucle* est une quantité qui dépend des variables du programme et qui est :

1. entière,
2. positive,
3. strictement décroissante.

Dans l'algorithme ci-dessus, la quantité m est un variant de boucle, en effet :

1. $m \in \mathbb{N}$ (par précondition).
2. m reste positif ou nul : par condition d'entrée dans la boucle $m \geq 1$, et comme on décrémente m de 1, m reste ≥ 0 après le tour de boucle.
3. m décroît strictement car il est diminué de 1 lors de chaque passage dans la boucle.

L'algorithme termine car on a trouvé un variant de boucle.

Q4– Donner la définition d'un invariant de boucle, puis prouver que cet algorithme est correct.

Un *invariant de boucle* est une propriété qui dépend des variables du programme et qui :

1. est vraie avant d'entrer dans la boucle (initialisation),
2. reste vraie après un tour de boucle si elle l'était au début du tour (conservation).

En sortie de boucle, la validité d'un invariant permet de prouver la correction de l'algorithme.

En notant m_0 la valeur initiale de m , montrons que la propriété I : « $r = (m_0 - m)n$ » est un invariant de boucle.

1. Avant d'entrer dans la boucle, $m = m_0$ donc $(m_0 - m)n = 0$. Comme r est initialisé à 0, la propriété I est vérifiée.
2. On suppose I vérifiée au début d'un tour de boucle et on note r' (resp. m') les valeurs prises par r (resp. m) à la fin de ce tour. On a $m' = m - 1$ et $r' = r + n$. Calculons : $(m_0 - m')n = (m_0 - (m - 1))n = (m_0 - m + 1)n = (m_0 - m)n + n$. Or, par hypothèse de récurrence (I vraie), $(m_0 - m)n = r$. Donc $(m_0 - m')n = r + n = r'$. La propriété I est donc conservée.

En sortie de boucle, la condition $m > 0$ est fausse, donc $m = 0$. L'invariant nous assure alors que $r = (m_0 - 0)n = m_0n$. L'algorithme calcule bien le produit attendu et est donc correct.

□ Exercice 2 : Retourner un tableau

Dans cet exercice, on s'intéresse à un algorithme permettant de « retourner » un tableau c'est-à-dire réorganiser l'ordre de ses éléments de façon à ce que le premier élément devienne le dernier, le second devienne l'avant-dernier et ainsi de suite. Par exemple, le tableau $\{2, 7, 1, 9, 3\}$ deviendrait $\{3, 9, 1, 7, 2\}$. En notant, t_0 le tableau initial, et t_r le tableau « retourné » on a donc pour tout $k \in \llbracket 0; n - 1 \rrbracket$, $t_r[k] = t_0[n - 1 - k]$. On propose pour cela l'algorithme suivant :

Algorithme : Retourner un tableau

Entrées : Un tableau t de taille n

Sorties : Aucune

Résultat : Le tableau t est modifié (retourné)

```

1  $i \leftarrow 0$ 
2  $j \leftarrow n - 1$ 
3 tant que  $j - i > 0$  faire
4   | échanger les éléments d'indice  $i$  et  $j$  dans  $t$ 
5   |  $i \leftarrow i + 1$ 
6   |  $j \leftarrow j - 1$ 
7 fin
```

Q5– Montrer que cet algorithme termine.

Dans le cas où le tableau est vide, l'algorithme termine directement sans entrer dans la boucle **while**. Sinon,

- $j - i$ est entier comme différence de deux entiers
- $j - i$ est positif avant d'entrer dans la boucle ($n \geq 1$) et reste positif par condition d'entrée dans la boucle
- $j - i$ décroît strictement puisque i est incrémenté et j décrémenté

Q6– Montrer que la propriété suivante notée I est un invariant de l'algorithme : $i + j = n - 1$.

- Avant d'entrer dans la boucle $i = 0$ et $j = n - 1$, donc I est vraie.
- On suppose I vraie au début d'un tour de boucle. Montrons qu'alors I est conservée à la fin de ce passage. En notant $i' = i + 1$ et $j' = j - 1$ les nouvelles valeurs, on a $i' + j' = i + 1 + j - 1 = i + j$. Or, par hypothèse $i + j = n - 1$, donc $i' + j' = n - 1$ et I est conservée.

Q7– Montrer que cet algorithme est correct, on pourra considérer l'invariant I' : « Le tableau t contient les valeurs de t_r (le tableau retourné) pour tous les indices $k \in \llbracket 0; i - 1 \rrbracket \cup \llbracket j + 1; n - 1 \rrbracket$ » et utiliser l'invariant I de la question précédente.

On vérifie que l'invariant proposé est vrai :

- Avant d'entrer dans la boucle $\llbracket 0; i - 1 \rrbracket \cup \llbracket j + 1; n - 1 \rrbracket = \emptyset$ et donc I' est vrai par vacuité.
- Conservé lors d'un passage dans la boucle : si I' est vrai en entrant dans la boucle, alors on échange $t[i]$ avec $t[j]$. D'après l'invariant I démontré à la question précédente, $j = n - 1 - i$. On échange donc $t[i]$ avec $t[n - 1 - i]$. Comme, par hypothèse, les éléments situés avant i et après j sont déjà correctement échangés (I' supposé vrai), la propriété I' reste vraie pour les indices s'étendant jusqu'à i et j .

En sortie de boucle, on a $j - i \leq 0$ et donc $\llbracket 0; i - 1 \rrbracket \cup \llbracket j + 1; n - 1 \rrbracket = \llbracket 0; n - 1 \rrbracket$, donc le tableau t contient bien les valeurs du tableau retourné.

Q8– On souhaite utiliser cet algorithme afin d'écrire en langage C une fonction `retourner_str` qui retourne une chaîne de caractères, c'est-à-dire que, par exemple, la chaîne "MP2I" devient "I2PM". Rappeler la façon dont sont implémentées en C les chaînes de caractères et expliquer pour quelle raison il n'est pas utile de fournir la longueur de la chaîne en paramètre à la fonction `retourner_str`.

En C, les chaînes de caractères sont des tableaux de caractères se terminant par le caractère sentinelle `'\0'`, on peut donc obtenir la taille d'une chaîne de caractères en recherchant la position de ce caractère, il n'est pas utile de la passer en paramètre.

Q9– Ecrire une fonction de signature `void echange(char s[], int i, int j)` qui échange dans s les caractères situés aux indices i et j .

```

1 void echange(char s[], int i, int j)
2 {
3     // echange les caractères situés aux indices i et j dans s
4     char temp = s[i];
5     s[i] = s[j];
6     s[j] = temp;
7 }
```

Q10– Ecrire une implémentation de la fonction `retourner_str` sans utiliser les fonctions de la bibliothèque `<string.h>`.

```

1 void retourner_str(char s[])
2 {
3     // Retourne en place la chaîne de caractère s
4     int n = 0;
5     while (s[n] != '\0')
6     {
7         n++;
8     }
9     int i = 0;
10    int j = n - 1;
11    while (j - i > 0)
12    {
13        echange(s, i, j);
14        i = i + 1;
15        j = j - 1;
16    }
17 }

```

Q11– On rappelle qu'un palindrome est un mot qui se lit de la même façon de droite à gauche ou de gauche à droite, par exemple « *radar* » est un palindrome, mais « *tata* » n'en est pas un. Pour tester si un mot est un palindrome un élève propose la solution suivante :

```

1 bool palindrome(char s[])
2 {
3     //renvoie true ssi s est un palindrome
4     char copie[] = s;
5     retourner_str(copie);
6     return s==copie;
7 }

```

Ce programme ne compile pas et produit une erreur à la ligne 4 : « *error : invalid initializer* ». Indiquer la source de cette erreur et expliquer comment la corriger, *on ne demande pas d'écrire une fonction corrigée de la fonction palindrome* mais d'indiquer de façon succincte la source de l'erreur commise à la ligne 4.

On ne peut pas affecter directement un tableau en C, on doit travailler élément par élément, on devrait donc utiliser une boucle **for** afin de copier un à un les caractères de *s* dans la chaîne *copie*.

Q12– On suppose corrigée l'erreur commise à la ligne 4, cette fonction est-elle correcte ? Justifier votre réponse.

On ne peut pas tester l'égalité des éléments de deux tableaux directement comme on tente de le faire à la ligne 6, on doit comparer un à un les éléments dans une boucle **for**.

❑ Exercice 3 : Recherche de minimums

On donne ci-dessous le programme d'un élève en C afin de rechercher le minimum d'un tableau d'entiers :

```

1  int min_eleve(int tab[], int taille)
2  {
3      // Renvoie le minimum des éléments de tab (supposé non vide)
4      assert(taille > 0);
5      int cmin = 0;
6      for (int i = 0; i < taille; i++)
7      {
8          if (tab[i] < cmin)
9          {
10             cmin = tab[i];
11          }
12      }
13      return cmin;
14  }

```

Q13– Proposer un test montrant que cette fonction ne répond pas à sa spécification.

Si le tableau ne contient que des valeurs *strictement positives* alors le minimum étant initialisé à 0 le test de la ligne 8 n'est jamais vrai et donc la valeur renvoyée pour le minimum sera 0. On peut par exemple prendre le tableau {2, 7, 5 } son minimum est 2 et la fonction renverra 0.

Q14– Corriger cette fonction afin de la rendre conforme à sa spécification.

```

1  int min_corrige(int tab[], int taille)
2  {
3      // Renvoie le minimum des éléments de tab (supposé non vide)
4      assert(taille > 0);
5      int cmin = tab[0];
6      for (int i = 1; i < taille; i++)
7      {
8          if (tab[i] < cmin)
9          {
10             cmin = tab[i];
11          }
12      }
13      return cmin;
14  }

```

On s'intéresse maintenant à la recherche des deux plus petites valeurs d'un tableau contenant au moins deux éléments. Pour cela on initialise deux valeurs `min1` et `min2` aux deux premières valeurs du tableau avec `min1 <= min2`, puis on parcourt le reste du tableau en mettant à jour ces valeurs en fonction de la valeur `tab[i]` rencontrée dans le tableau.

Q15– Expliquer succinctement, comment mettre à jour `min1` et `min2` de façon à préserver l'invariant suivant : `min1` et `min2` sont les deux plus petites du sous tableau `tab[0] ... tab[i-1]` et `min1 <= min2` (on pourra distinguer les cas où `tab[i]` est inférieur à `min1` ou compris entre `min1` et `min2`).

- Si `tab[i] < min1` alors `min2` prend la valeur de `min1` et `min1` celle de `tab[i]`
- Si `min1 <= tab[i] < min2` alors `min2` prend la valeur de `tab[i]`
- Sinon `tab[i]` n'est pas l'un des deux minimums, on ne fait rien.

Q16– On propose pour implémenter cette fonction en C, d'utiliser un type structuré `couple` contenant deux valeurs de type `int`. Donner la définition de ce type structuré qu'on appellera `couple` et dont les champs seront appelés `premier` et `second`.

```
1 struct couple_s
2 {
3     int premier;
4     int second;
5 };
6 typedef struct couple_s couple;
```

Q17– Écrire la fonction de signature `couple deuxmin(int tab[], int n)` qui prend en argument un tableau et sa taille et renvoie ses deux plus petites valeurs dans une variable de type `couple`.

```
1 couple deuxmin(int tab[], int taille)
2 {
3     assert(taille > 1);
4     couple min;
5     if (tab[0] < tab[1])
6     {
7         min.premier = tab[0];
8         min.second = tab[1];
9     }
10    else
11    {
12        min.premier = tab[1];
13        min.second = tab[0];
14    }
15    for (int i = 2; i < taille; i++)
16    {
17        if (tab[i] < min.premier)
18        {
19            min.second = min.premier;
20            min.premier = tab[i];
21        }
22        else if (tab[i] < min.second)
23        {
24            min.second = tab[i];
25        }
26    }
27    return min;
28 }
```

Q18– Une autre possibilité d'implémentation consiste à passer en paramètre deux pointeurs vers des entiers qui seront modifiés dans la fonction et à ne rien renvoyer. Donner alors la signature de la fonction ainsi que son implémentation.

```

1 void deuxmin2(int tab[], int taille, int* min1, int* min2)
2 {
3     assert(taille > 1);
4     if (tab[0] < tab[1])
5     {
6         *min1 = tab[0];
7         *min2 = tab[1];
8     }
9     else
10    {
11        *min1 = tab[1];
12        *min2 = tab[0];
13    }
14    for (int i = 2; i < taille; i++)
15    {
16        if (tab[i] < *min1)
17        {
18            *min2 = *min1;
19            *min1 = tab[i];
20        }
21        else if (tab[i] < *min2)
22        {
23            *min2 = tab[i];
24        }
25    }
26 }

```

□ **Exercice 4** : Plus long plateau d'un tableau

Un « plateau » dans un tableau est une séquence d'éléments contigus tous égaux. Par exemple, dans le tableau {1, 2, 2, 2, 3, 3, 1}, la séquence 2, 2, 2 est un plateau de longueur 3.

Q19– Déterminer la longueur du plus long plateau pour le tableau $t = \{4, 4, 1, 1, 1, 1, 2, 2\}$.

Le plus long plateau est constitué de la suite de 1, sa longueur est de 4.

Q20– Écrire une fonction C `int max_plateau(int tab[], int n)` qui prend en paramètre un tableau non vide de taille `n` et renvoie la longueur de son plus long plateau.

```

1 int max_plateau(int tab[], int n) {
2     int max_len = 1;
3     int len_actuelle = 1;
4     for (int i = 1; i < n; i++) {
5         if (tab[i] == tab[i-1]) {
6             len_actuelle++;
7         } else {
8             len_actuelle = 1;
9         }
10        if (len_actuelle > max_len) {
11            max_len = len_actuelle;
12        }
13    }
14    return max_len;
15 }

```

Q21– Quelle est la complexité temporelle de cette fonction ? Justifiez.

La complexité est en $\mathcal{O}(n)$. En effet, l'algorithme parcourt le tableau une seule fois à l'aide d'une boucle `for` exécutant $n - 1$ itérations. Chaque itération n'effectue qu'un nombre constant d'opérations élémentaires (comparaisons de valeurs et d'indices, incrémentations, affectations).

- Q22**– On souhaite désormais, en plus de la longueur du plus long plateau, connaître le nombre de fois où cette longueur maximale apparaît dans le tableau. Écrire une fonction C `void info_plateau(int tab[], int n, int *max_len, int *occ)` qui affecte à la variable pointée par `max_len` la longueur du plus long plateau, et à la variable pointée par `occ` le nombre de plateaux ayant cette longueur maximale.

```
1 void info_plateau(int tab[], int n, int *max_len, int *occ) {
2     *max_len = 1;
3     *occ = 1;
4     int len_actuelle = 1;
5     for (int i = 1; i < n; i++) {
6         if (tab[i] == tab[i-1]) {
7             len_actuelle++;
8         } else {
9             len_actuelle = 1;
10        }
11
12        if (len_actuelle > *max_len) {
13            *max_len = len_actuelle;
14            *occ = 1;
15        } else if (len_actuelle == *max_len) {
16            *occ = *occ + 1;
17        }
18    }
19 }
```